Deep Neural Networks – Programming Assignment 2

RNN, LSTM, GRU and Transformer

Name: Souvik Dutta Student ID: 2026AJ05991

Settings used: hidden size h = 64 for RNN/LSTM/GRU (Problem 1); Transformer width d = 128 with 4 heads and feed-forward width 512 (2 encoder layers for HAR, 4 layers for the language model); Adam, learning rate 10^{-3}, batch size 128, 12 epochs; seeds 42, 43, 44; weights after the final epoch (no early stopping). Run on Google Colab (Tesla T4, PyTorch 2.14.1 + CUDA 13.0).

1 Problem 1 – RNN, LSTM and GRU

1.1 Setup

UCI HAR raw inertial signals are used as given: 9 channels, 128 time steps per window, 6 classes, 7,352 training and 2,947 test windows (21 and 9 subjects, no overlap). For each seed, 4 of the 21 training subjects are held out for validation with *np.random.default_rng(seed).choice*, so the validation split changes with the seed. Per-channel mean and standard deviation come from the remaining training subjects only. The classes are close to balanced (Figure 1: about 800 windows for walking downstairs to about 1,130 for laying), so accuracy and macro-F1 track each other. The adding problem uses the provided generator (10k/1k/1k sequences, seeds 0/1/2) with an assert that every sequence has exactly two markers, one in each half. Its constant-prediction baseline on the test set is 0.1754, 0.1697, 0.1679 and 0.1527 for T = 50, 100, 200, 400, and a run counts as *solved* if test MSE < 0.05. The models in the experiments are one recurrent layer (torch.nn RNN/LSTM/GRU) plus a linear head on the last hidden state.

1.2 Cells from scratch

MyRNNCell, MyLSTMCell and MyGRUCell use only matrix products and element-wise operations, keep PyTorch's weight layout and gate order (i,f,g,o and r,z,n), and are checked in *tests/test_models.py* against the torch.nn cells after copying the weights (absolute tolerance 10^{-5}, LSTM cell state included). **Parameter counts.** One gate needs W (h×d), U (h×h) and a bias, so for h = 64, d = 9: $64{\cdot}9 + 64^2 + 64 = 4{,}736$ (RNN), $4 \times 4{,}736 = 18{,}944$ (LSTM), $3 \times 4{,}736 = 14{,}208$ (GRU). PyTorch stores separate b_{ih} and b_{hh}, which adds h per gate and gives 4,800, 19,200 and 14,400; with the $64{\cdot}6 + 6 = 390$ parameters of the head these become the model sizes in Table 1 (5,190 / 19,590 / 14,790).

1.3 HAR classification

Table 1. HAR test results, h = 64, same 12-epoch budget; mean ± sample std over seeds 42, 43, 44. Time is training seconds per epoch.

Model	Test accuracy (%)	Macro-F1 (%)	Parameters	s / epoch
RNN	65.74 ± 3.33	64.99 ± 3.12	5,190	0.19
LSTM	86.00 ± 2.95	85.66 ± 3.45	19,590	0.18
GRU	88.85 ± 0.67	88.84 ± 0.72	14,790	0.18

The GRU is the most accurate and the most stable (std 0.7 points); the LSTM is about 3 points lower with a std of 3.0, and the plain RNN is far behind (65.7 %). Cost per epoch is almost the same for the three cells. The confusion matrix of the run with the highest test accuracy (GRU, seed 44, 89.6 %; Figure 2, shown for illustration only) has 306 errors. 183 of them (60 %) are sitting ↔ standing confusions (89 + 94): the two postures differ mainly in body orientation, which raw accelerations show only weakly. Laying is almost perfect (519/537), and the three walking classes account for most of the remaining errors.

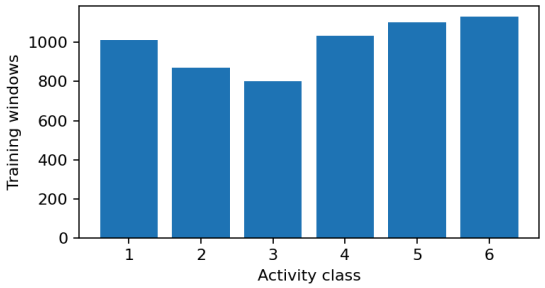


Figure 1. Subject-wise HAR training class balance (classes 1–6).

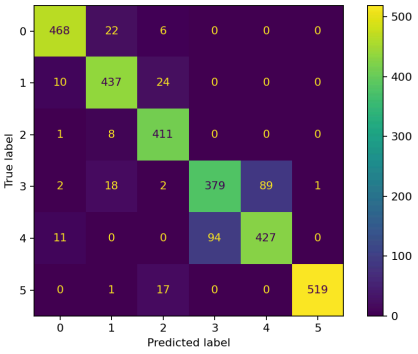


Figure 2. Confusion matrix, GRU seed 44 (0–2 walking, 3 sitting, 4 standing, 5 laying).

1.4 Adding problem

Table 2. Adding problem, test MSE (mean ± std over 3 seeds; solved runs in brackets). Last column: s/epoch for RNN / LSTM / GRU.

T	Baseline	RNN	LSTM	GRU	s / epoch
50	0.1754	0.1746 ± 0.0004 (0/3)	0.1704 ± 0.0029 (0/3)	0.0096 ± 0.0018 (3/3)	0.24 / 0.26 / 0.24
100	0.1697	0.1697 ± 0.0003 (0/3)	0.1696 ± 0.0004 (0/3)	0.1332 ± 0.0317 (0/3)	0.24 / 0.27 / 0.26
200	0.1679	0.1681 ± 0.0003 (0/3)	0.1683 ± 0.0006 (0/3)	0.1672 ± 0.0021 (0/3)	0.25 / 0.29 / 0.29
400	0.1527	0.1537 ± 0.0007 (0/3)	0.1534 ± 0.0002 (0/3)	0.1526 ± 0.0005 (0/3)	0.24 / 0.43 / 0.47

Only the GRU solves anything, and only at T = 50 (3/3 runs, MSE 0.0096). At T = 100 its three runs end at 0.098, 0.144, 0.158, i.e. it leaves the plateau in two of the three seeds but never reaches 0.05. For T ≥ 200 every model sits on the baseline (Figure 3 and the validation curves in Figure 4). The RNN and the LSTM never leave the baseline, even at T = 50, which for the LSTM is a consequence of the small 12-epoch budget rather than of a capacity limit. Time per epoch stays at about 0.24 s for the RNN but grows for the LSTM and GRU at T = 400 (0.43 and 0.47 s).

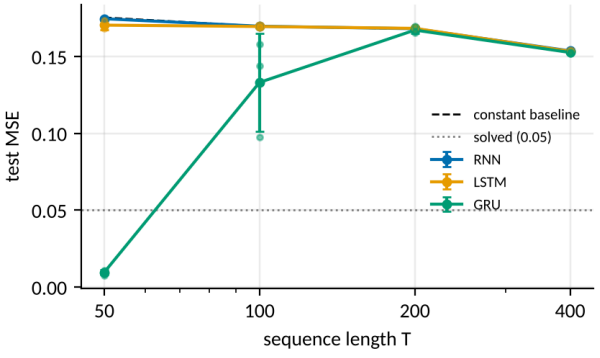


Figure 3. Test MSE against sequence length (mean ± std, dots are seeds).

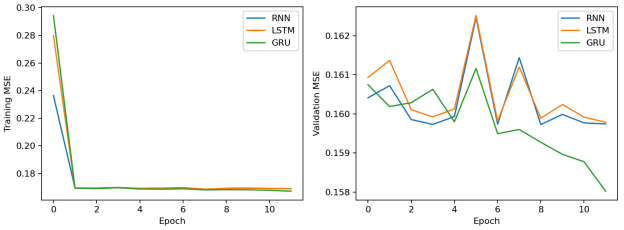


Figure 4. T = 200 training (left) and validation (right) MSE per epoch.

1.5 Analysis

(a) Memory and gradients. Figure 5 shows the norm of the loss gradient with respect to the input at step t, $\|\partial L/\partial x_t\|$, for T = 100 (a proxy for $\partial L/\partial h_t$; it is what the code logs). Before training the gradient from the end of the sequence is about 10^{-5}–10^{-4} but only about 10^{-21}–10^{-23} at t = 0 (and underflows to zero for the LSTM and GRU at the first steps), so early inputs cannot influence the loss. After training the RNN profile is unchanged, the LSTM improves to about 10^{-16} at t = 0, and the GRU is flat at about 10^{-5} over all 100 steps. This matches Table 2: the GRU is the only cell that learns anything at T ≥ 50. The result for the LSTM is probably a budget effect – its gradient path exists, but 12 epochs at lr 10^{-3} do not suffice to exploit it.

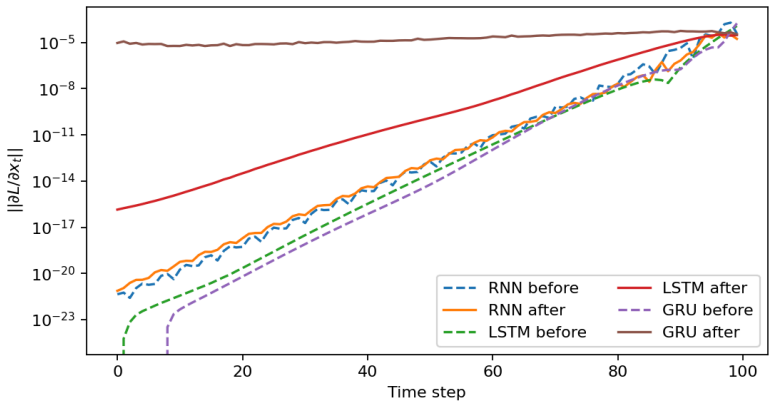


Figure 5. $\|\partial L/\partial x_t\|$ against t before (dashed) and after (solid) training, T = 100, log scale.

(b) Hidden size. Accuracy rises from h = 16 to h = 64 for both cells (GRU 83.7 → 88.9 %, LSTM 72.1 → 86.0 %) and then stops: h = 128 has about 3.7× the parameters (LSTM 71,942, GRU 54,150) and is not better (GRU 88.3 %, LSTM 85.5 %; the differences are within one std, Figure 6). The GRU is better than the LSTM at every size, so h = 32 is already enough for it (88.1 % with 4,326 parameters).

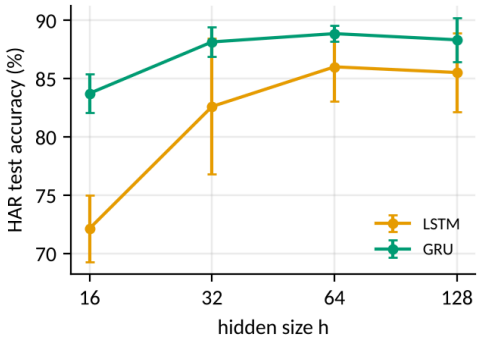


Figure 6. HAR accuracy against hidden size (mean ± std, 3 seeds).

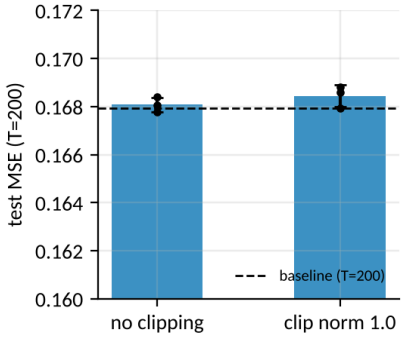


Figure 7. Gradient clipping, RNN, T = 200 (3 seeds).

(c) Individual ablation – gradient clipping (norm 1.0) for the RNN at T = 200. (The individualisation table was withdrawn in the course clarification, so the sample ablation was used.) Test MSE is 0.1681 ± 0.0003 without clipping and 0.1684 ± 0.0004 with clipping, against a baseline of 0.1679 (Figure 7). Both settings sit on the baseline, so the ablation is *inconclusive*: the difference (0.0004) is the size of the seed-to-seed spread, and clipping cannot help when the problem is vanishing rather than exploding gradients – it caps large norms but cannot amplify the 10^{-20}-sized signal that reaches the early steps.

(d) Recommendation for a wearable activity tracker. I would ship the **GRU**: best accuracy (88.8 ± 0.7 %), lowest variance, about 25 % fewer parameters than the LSTM (14,790 vs 19,590) at the same training cost, and a gradient that reaches early steps. If memory or battery dominates, GRU h = 32 (4,326 parameters, 88.1 ± 1.3 %) loses under one point. The main residual weakness is sitting vs standing (60 % of the errors), which needs a better orientation cue rather than a bigger model. Limits: 12 epochs, 3 seeds and one dataset.

2 Problem 2 – Transformer

2.1 Scaled dot-product attention

Attention is implemented as $\text{softmax}(QK^T/\sqrt{d_k} + \text{mask})V$. The unit tests check that (i) every row of the weights sums to 1, (ii) the output is compared with *F.scaled_dot_product_attention*, and (iii) with a causal mask the weights above the diagonal are exactly 0. **Hand-checkable example** ($d_k = 2$): Q = [[1,0],[0,1],[1,1]], K = [[1,1],[1,0],[0,1]], V = [[1,2],[3,4],[5,6]]. $QK^T/\sqrt{2}$ = [[.7071, .7071, 0], [.7071, 0, .7071], [1.4142, .7071, .7071]]. For row 1, $e^{.7071}$ = 2.0281, so the weights are 2.0281/5.0562 = 0.4011, 0.4011 and 1/5.0562 = 0.1978, and the output is 0.4011·[1,2] + 0.4011·[3,4] + 0.1978·[5,6] = [2.593, 3.593]. The code gives the

same values for all three rows (outputs [2.593, 3.593], [3.000, 4.000], [2.490, 3.490]); the weights are shown in Figure 8.

2.2 Multi-head attention and encoder block

The block is pre-LayerNorm: x + Dropout(MHA(LN(x))), then x + Dropout(FFN(LN(x))) with dropout 0.1. MHA has four d×d projections (W_Q, W_K, W_V, W_O) split into h heads of size d/h = 32; the FFN is 128 → 512 → 128 with GELU. **Parameters of one block (d = 128, d_{ff} = 512):** weights $4d^2 + 2 \cdot d \cdot d_{ff}$ = 65,536 + 131,072 = 196,608; biases $4d + d_{ff} + d$ = 1,152; two LayerNorms 4d = 512; total 198,272. The 4-layer language model adds the embedding (65·128 = 8,320), a final LayerNorm (256) and the output layer (8,385): 4 · 198,272 + 8,320 + 256 + 8,385 = 810,049, which equals the count in results.csv. The positional encoding is fixed sinusoidal and has no parameters.

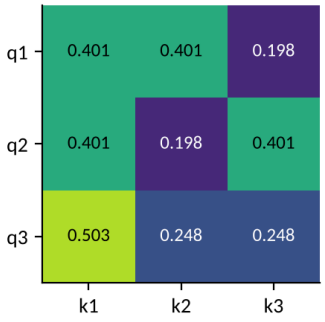


Figure 8. Attention weights of the hand example.

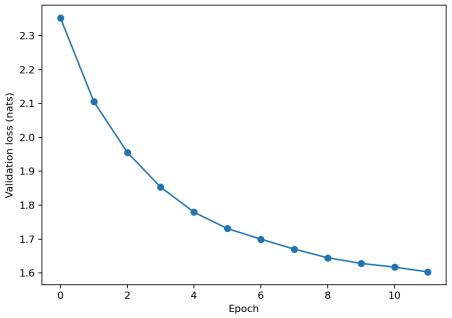


Figure 9. Tiny Shakespeare validation loss (nats) per epoch.

2.3 Transformer encoder on HAR

The classifier projects each 9-channel time step to d = 128 (128 tokens + a CLS token), applies 2 encoder blocks with 4 heads and classifies from the CLS output (415,494 parameters).

Table 3. HAR test results by positional information (mean ± std, seeds 42, 43, 44).

Positional encoding	Test accuracy (%)	Macro-F1 (%)
none	89.16 ± 1.16	89.07 ± 1.13
sinusoidal (fixed)	88.21 ± 0.68	88.09 ± 0.73
learned	88.54 ± 1.20	88.37 ± 1.24

The three variants are within about one point of each other and within one std, so this experiment does not show that positional information matters on HAR; the model without any position still classifies as well, which suggests that most of the signal is in the distribution of the values in a window (e.g. gravity direction and amplitude) rather than in their order. Compared with the GRU (88.9 % with 14,790 parameters) the Transformer is comparable in accuracy (88.2–89.2 %) but has about 28× more parameters and is much slower: the whole 9-run cell took 188 s, about 1.7 s per epoch including evaluation, against 0.18 s of training per epoch for the GRU. (Per-epoch time was not logged separately for the Transformer, so this is a coarse figure.)

2.4 Character-level language model

Tiny Shakespeare, 65 characters, context 128, windows with stride 64, a 4-layer causal model (810,049 parameters), Adam 10^{-3}, batch 128, 12 epochs, one run (seed 42). Validation loss falls from about 2.35 to **1.604 nats = 2.31 bits per character** (a uniform guess gives ln 65 = 4.17 nats = 6.02 bits). The curve (Figure 9) is still going down by about 0.01 per epoch at the end, so the model is under-trained rather than over-fitted. Samples (500 characters, top-k 20) show the effect of temperature: at **0.2** the text is grammatical but repetitive ("the state of the proclaim of the commons, / That the world of the prince of the love"); at **0.8** the play format is intact (speaker names in capitals, line breaks) and most words are real, with some blends such as "restimed" and "lookss"; at **1.5** many tokens are non-words and the punctuation breaks down ("stuppricing", "sosengetet"). Low temperature trades diversity for safe, repeated phrases; high temperature does the reverse.

2.5 Analysis and ablations

(a) Attention heatmaps. The saved heatmap was produced without the causal mask, so it does not show the attention the trained model actually uses. It is left out and will be added here after re-running with the mask.

(b) Number of heads. The head ablation measures only the cost of one forward pass of an untrained one-layer model (d = 128) with 1, 4 and 8 heads on random tokens. The parameter count is identical, 215,233, because the four d×d projections do not depend on the number of heads (the heads only split d into d/h parts); the forward time is also the same (1.4 ms and 1.2 ms for 4 and 8 heads; the 114 ms of the first run is CUDA start-up). The effect of the head count on validation loss was *not* measured.

(c) Causal mask. I did not train a model without the mask, so there is no loss or sample to report. The measured part is cost: the unmasked forward pass takes the same time as the masked one (Table 4), because the mask only changes values and the full n×n score matrix is computed either way.

(d) Context length and $O(n^2d)$. Table 4 gives forward time and peak GPU memory for a one-layer model, batch 2. Times are 1–1.7 ms with no trend (the run is dominated by kernel-launch overhead), so the time scaling is *not* demonstrated. Peak memory grows from 18.8 MB to 39.1 MB; the increase per doubling of n is 1.2, 4.2 and 14.9 MB, i.e. ×3.5 and ×3.5 per step, approaching the ×4 expected for the n×n score matrices (batch × heads × n^2 floats = 8.4 MB per n×n tensor at n = 512). QK^T and the weighted sum with V each cost $O(n^2 \cdot d)$ operations per layer and the weights need $O(n^2)$ memory per head, which is why long contexts are expensive; the feed-forward and projection layers are only linear in n.

Table 4. Forward pass of an untrained one-layer model, batch 2, random tokens.

Context n	Causal (ms)	Unmasked (ms)	Peak memory (MB)
64	1.66	1.08	18.8
128	1.15	1.14	20.0
256	1.39	1.05	24.3
512	1.48	1.05	39.1

AI-use statement

Tools: Claude (Anthropic). AI assistance supported code structuring, debugging, test design and plotting, and was used to review the results against the assignment text and to draft and format this report. All experiments were run by me on Colab; every number in this report was recomputed from *results.csv* and the output CSV files, and the parameter counts and the hand-attention example were re-derived independently. Anything not run is stated as not run (the attention heatmaps with the causal mask, and the effect of the head count and of removing the mask on the loss).

References

[1] PyTorch documentation, pytorch.org/docs/stable (accessed Nov 2026). [2] D. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones”, ESANN 2013; UCI Machine Learning Repository, archive.ics.uci.edu/dataset/240 (accessed Nov 2026). [3] A. Karpathy, char-rnn – Tiny Shakespeare, github.com/karpathy/char-rnn (accessed Nov 2026). [4] A. Vaswani et al., “Attention Is All You Need”, NeurIPS 2017.